

Project Submission – Database Administration

Vehicle Service Management Database

Student Details

Student Name	PRADEEPA.S
Register Number	2428C0385
Department	BSc AI&ML
ID	asbrubw2428c0385
College	Akshaya College of Arts and Science
Date of Submission	11-09-2026

1. Problem Statement & Objectives – 5 Marks

Problem Statement

Vehicle service centers handle a large number of customers, vehicles, and repeat service visits every day. When this information is maintained manually or in scattered spreadsheets, it becomes difficult to track which vehicle belongs to which customer, what services have been performed, when a vehicle is due for its next service, and how much revenue has been generated. This often leads to lost records, duplicate entries, delays in retrieving service history, and poor customer service.

The Vehicle Service Management Database is designed to solve this problem by storing customer, vehicle, and service information in a structured, relational database. It allows the service center staff to quickly register customers and their vehicles, log on to every service performed, track the status of ongoing services, and generate reports such as pending services, total revenue, and service history for any vehicle.

Objectives

- To design a normalized relational database for a vehicle service center.
- To maintain accurate records of customers, their vehicles, and service history.
- To reduce data redundancy by linking related information through primary and foreign keys.
- To enable quick retrieval of information using SQL queries (filtering, sorting, searching).
- To generate useful summaries such as total services, revenue, and pending work using aggregate functions.
- To demonstrate the use of joins to combine data from multiple related tables.

2. Database Design – 10 Marks

2.1 Database Structure

The database consists of three related tables — Customer, Vehicle, and Service. A Customer can own multiple Vehicles, and each Vehicle can have multiple Service records over time. This one-to-many relationship structure avoids data duplication and keeps the database normalized.

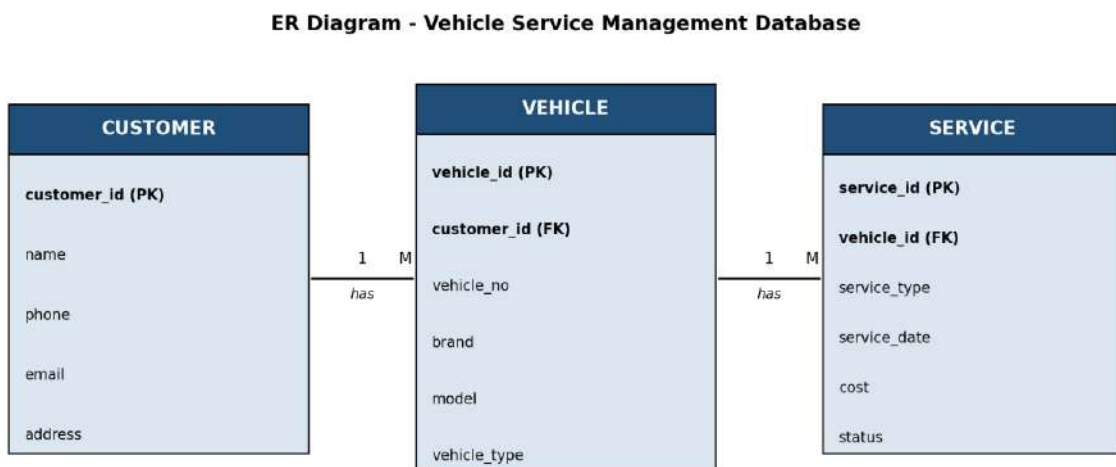
Tables Used

Table Name	Purpose
Customer	Stores customer personal and contact details
Vehicle	Stores vehicle details linked to each customer
Service	Stores the service history, cost, and status of each vehicle

2.2 Entity Relationship Diagram

The diagram below shows the three tables, their attributes, primary keys (PK), foreign keys (FK), and the one-to-many relationships between them.

Diagram:



2.3 Technologies Used

Technology / Tool	Purpose
Oracle Database	Relational database management system used to store and manage data
SQL	Used for database creation, data manipulation, and querying
SQL Developer / SQL*Plus	Used to write, execute, and test SQL statements

3. Practical Implementation – 15 Marks

3.1 Database and Table Creation

Database Creation

The database was implemented in Oracle. Since Oracle does not use a CREATE DATABASE statement (each user works within their own schema), the tables were created directly using CREATE TABLE with IDENTITY columns for auto-generated primary keys.

Customer Table

```
CREATE TABLE Customer (
  customer_id  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name         VARCHAR2(50) NOT NULL,
  phone       VARCHAR2(15) NOT NULL UNIQUE,
  email        VARCHAR2(50),
  address      VARCHAR2(100)
);
```

Vehicle Table

```
CREATE TABLE Vehicle (
  vehicle_id  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id NUMBER NOT NULL,
  vehicle_no  VARCHAR2(20) NOT NULL UNIQUE,
  brand       VARCHAR2(30) NOT NULL,
  model      VARCHAR2(30) NOT NULL,
  vehicle_type VARCHAR2(20),
  CONSTRAINT fk_vehicle_customer FOREIGN KEY (customer_id)
    REFERENCES Customer(customer_id) ON DELETE CASCADE
);
```

Service Table

```
CREATE TABLE Service (
  service_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  vehicle_id NUMBER NOT NULL,
  service_type VARCHAR2(30) NOT NULL,
  service_date DATE NOT NULL,
  cost NUMBER(10,2) NOT NULL,
  status VARCHAR2(20) DEFAULT 'Pending',
  CONSTRAINT fk_service_vehicle FOREIGN KEY (vehicle_id)
    REFERENCES Vehicle(vehicle_id) ON DELETE CASCADE
);
```

3.2 Constraints and Relationships

Table	Primary Key	Foreign Key	Other Constraints
Customer	Customer id	None	NOT NULL (name, phone), UNIQUE (phone)
Vehicle	Vehicle id	customer_id -> Customer(customer_id)	NOT NULL, UNIQUE (vehicle_no)
Service	Service id	vehicle_id -> Vehicle(vehicle_id)	NOT NULL, DEFAULT ('Pending' for status)

3.3 Data Insertion

Customer Data

```
INSERT INTO Customer (name, phone, email, address)
VALUES ('Ramesh Kumar', '9876543210', 'ramesh@gmail.com', 'Salem');
-- (repeated for Priya Suresh, Arjun Raj, Divya Manoharan, Karthik Selvam)
```

Vehicle Data

```
INSERT INTO Vehicle (customer_id, vehicle_no, brand, model, vehicle_type)
VALUES (1, 'TN30AB1234', 'Honda', 'City', 'Car');
-- (repeated for each customer's vehicle)
```

Service Data

```
INSERT INTO Service (vehicle_id, service_type, service_date, cost, status)
VALUES (1, 'Oil Change', DATE '2026-01-05', 1200.00, 'Completed');
-- (repeated for each service record)
COMMIT;
```

3.4 SQL Operation

INSERT

Add a new record into a table.

```
INSERT INTO Customer (name, phone, email, address)  
VALUES ('Vignesh Babu', '9876543215', 'vignesh@gmail.com', 'Namakkal');
```

SELECT

Retrieves records from a table.

```
SELECT * FROM Service;
```

UPDATE

Modifies existing records.

```
UPDATE Service SET status = 'Completed' WHERE service_id = 4;
```

DELETE

Removes records from a table.

```
DELETE FROM Customer WHERE name = 'Vignesh Babu';
```

WHERE

Filters records based on a condition.

```
SELECT * FROM Service WHERE status = 'Pending';
```

ORDER BY

Sorts the result set.

```
SELECT * FROM Service ORDER BY cost DESC;
```

LIKE

Searches using a partial value/pattern.

```
SELECT * FROM Customer WHERE name LIKE 'K%';
```

BETWEEN

Retrieves records within a specified range.

```
SELECT * FROM Service
WHERE service_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31';
```

Aggregate Functions

COUNT(), SUM(), AVG(), MAX(), and MIN() summarize data across rows.

```
SELECT COUNT(*) AS total_services FROM Service;
SELECT SUM(cost) AS total_revenue FROM Service;
SELECT AVG(cost) AS average_cost FROM Service;
SELECT MAX(cost) AS highest_cost FROM Service;
SELECT MIN(cost) AS lowest_cost FROM Service;
```

GROUP BY

Groups service records by a column, e.g. status.

```
SELECT status, COUNT(*) AS total_count
FROM Service
GROUP BY status;
```

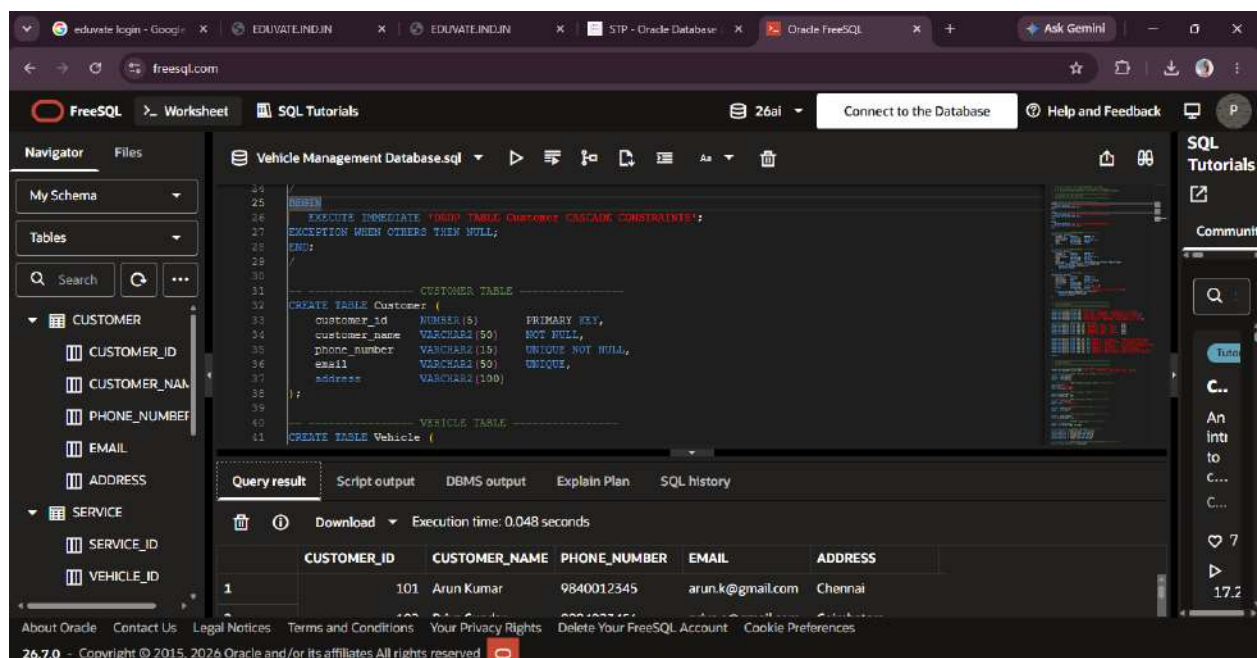
INNER JOIN

Combines related rows from multiple tables.

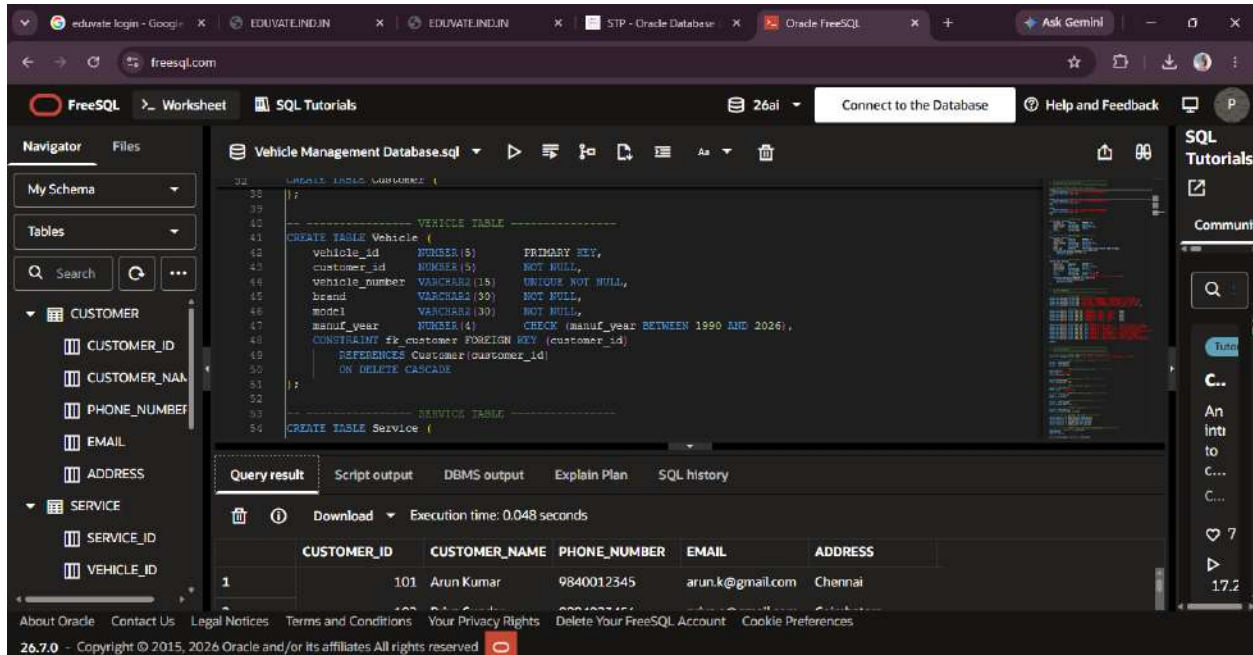
```
SELECT c.name, v.vehicle_no, s.service_type, s.service_date, s.cost, s.status
FROM Customer c
INNER JOIN Vehicle v ON c.customer_id = v.customer_id
INNER JOIN Service s ON v.vehicle_id = s.vehicle_id
ORDER BY s.service_date;
```

3.5 Project Execution

The database was created in Oracle by first defining the Customer, Vehicle, and Service tables with appropriate primary and foreign key constraints. Sample data was then inserted into each table in the correct order (Customer, Vehicle, then Service) so that foreign key references were always valid. SQL queries were executed one at a time in SQL Developer/SQL*Plus to create the schema, insert data, and then demonstrate each SQL operation (INSERT, SELECT, UPDATE, DELETE, filtering, sorting, aggregation, and joins), with the output of each query verified before moving to the next step.



Screenshot 3 – Vehicle Table with Records



The screenshot shows the Oracle FreeSQL interface with the 'Vehicle Management Database.sql' file open. The SQL script defines the 'Vehicle' table with the following structure:

```

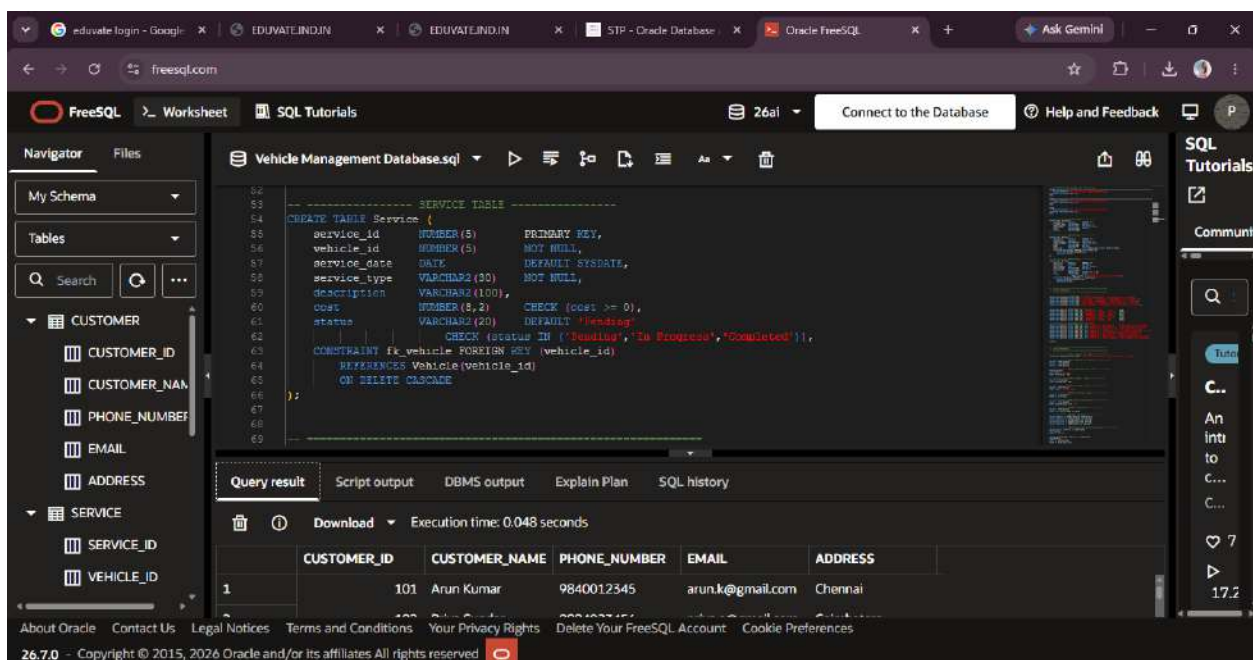
CREATE TABLE Vehicle (
  vehicle_id NUMBER(5) PRIMARY KEY,
  customer_id NUMBER(5) NOT NULL,
  vehicle_number VARCHAR2(15) UNIQUE NOT NULL,
  brand VARCHAR2(30) NOT NULL,
  model VARCHAR2(30) NOT NULL,
  manuf_year NUMBER(4) CHECK (manuf_year BETWEEN 1990 AND 2026),
  CONSTRAINT fk_customer FOREIGN KEY (customer_id)
    REFERENCES Customer (customer_id)
    ON DELETE CASCADE
);

```

The 'Query result' tab shows the execution of the script, resulting in the following table structure and data:

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
101	Arun Kumar	9840012345	arunk@gmail.com	Chennai

Screenshot 4 – Service Table with Records



The screenshot shows the Oracle FreeSQL interface with the 'Vehicle Management Database.sql' file open. The SQL script defines the 'Service' table with the following structure:

```

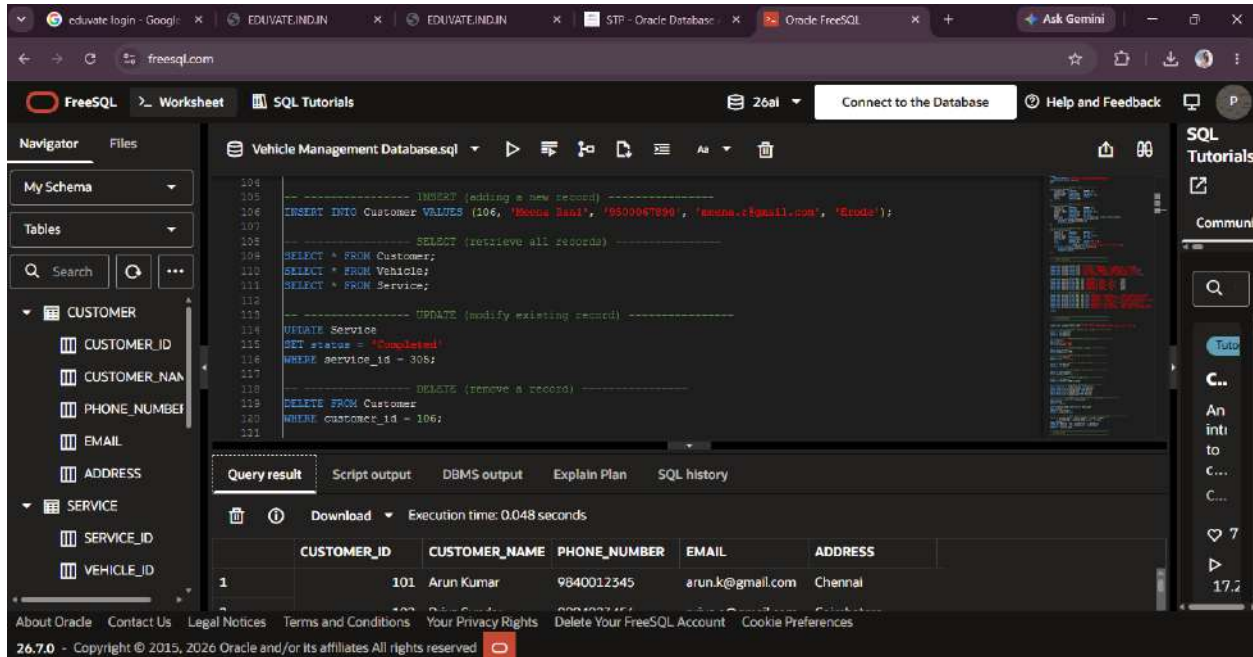
CREATE TABLE Service (
  service_id NUMBER(5) PRIMARY KEY,
  vehicle_id NUMBER(5) NOT NULL,
  service_date DATE DEFAULT SYSDATE,
  service_type VARCHAR2(30) NOT NULL,
  description VARCHAR2(100),
  cost NUMBER(8,2) CHECK (cost >= 0),
  status VARCHAR2(20) DEFAULT 'Pending',
  CHECK (status IN ('Pending', 'In Progress', 'Completed')),
  CONSTRAINT fk_vehicle FOREIGN KEY (vehicle_id)
    REFERENCES Vehicle (vehicle_id)
    ON DELETE CASCADE
);

```

The 'Query result' tab shows the execution of the script, resulting in the following table structure and data:

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
101	Arun Kumar	9840012345	arunk@gmail.com	Chennai

Screenshot 5 – INSERT / UPDATE / DELETE Operations



The screenshot shows the FreeSQL web interface with the following SQL script in the editor:

```

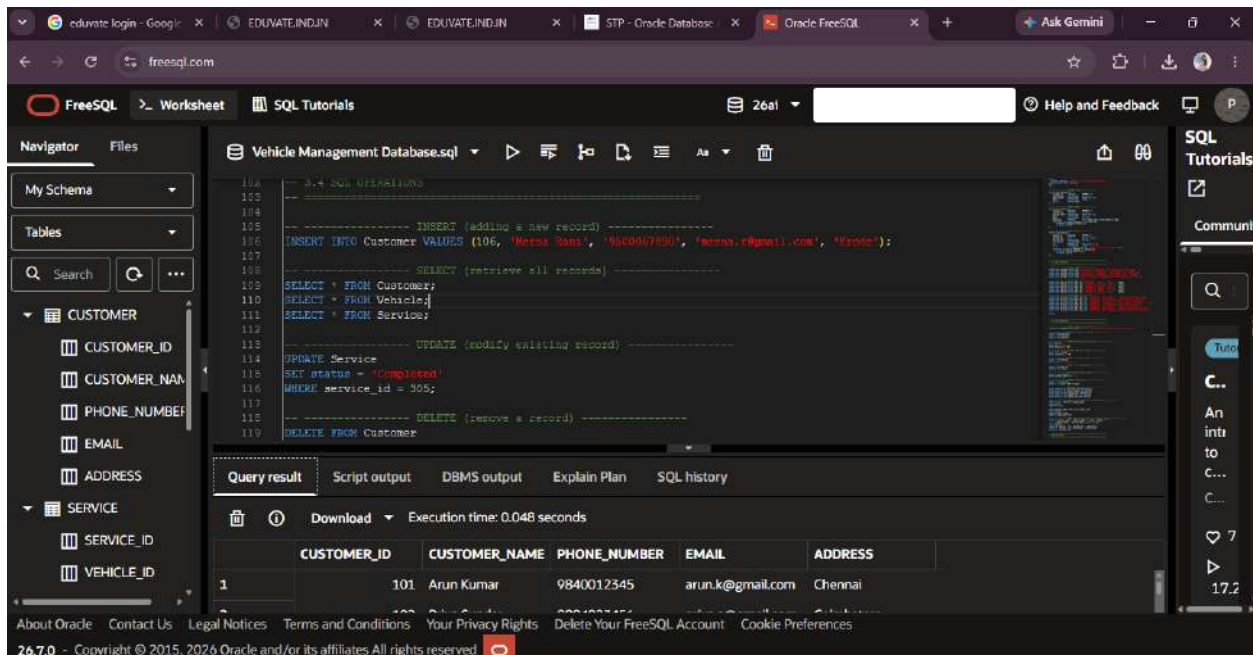
104 ----- INSERT (adding a new record) -----
105 INSERT INTO Customer VALUES (106, 'Kenna Bani', '9500067890', 'kenna.r@gmail.com', 'Kenna');
106 ----- SELECT (retrieve all records) -----
107 SELECT * FROM Customer;
108 SELECT * FROM Vehicle;
109 SELECT * FROM Service;
110 ----- UPDATE (modify existing record) -----
111 UPDATE Service
112 SET status = 'Completed'
113 WHERE service_id = 305;
114 ----- DELETE (remove a record) -----
115 DELETE FROM Customer
116 WHERE customer_id = 106;
117

```

The Query result table shows the following data:

	CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai

Screenshot 6 – SELECT and Filtering Operations



The screenshot shows the FreeSQL web interface with the following SQL script in the editor:

```

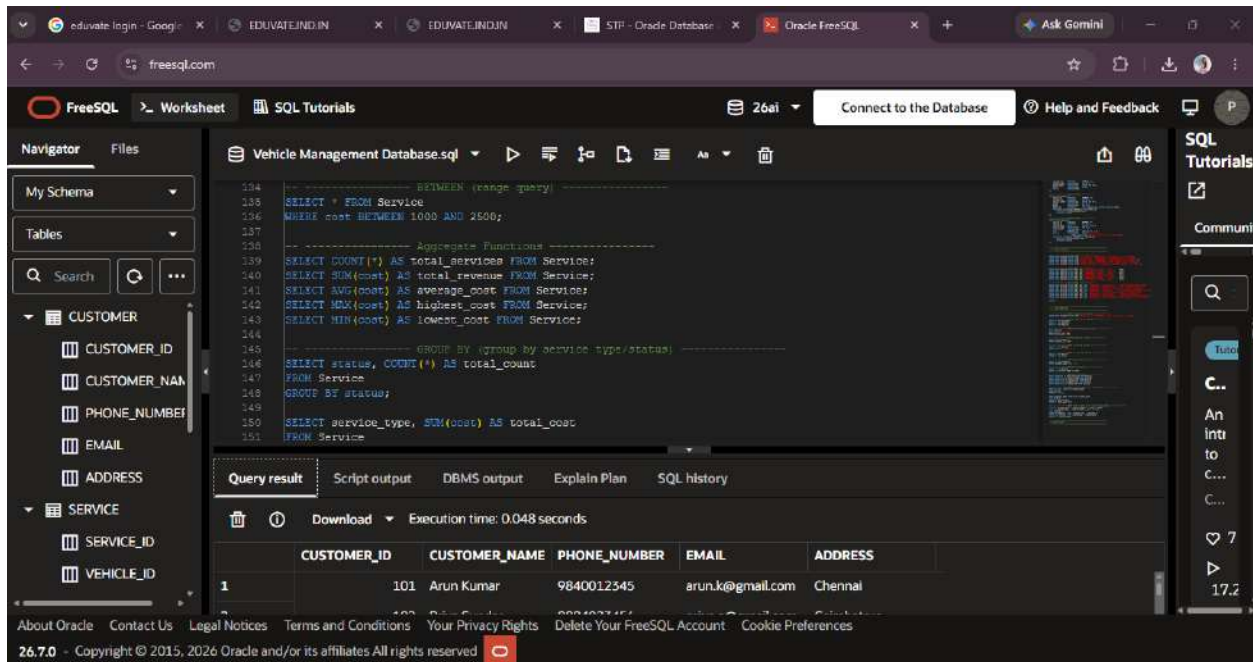
104 ----- INSERT (adding a new record) -----
105 INSERT INTO Customer VALUES (106, 'Kenna Bani', '9500067890', 'kenna.r@gmail.com', 'Kenna');
106 ----- SELECT (retrieve all records) -----
107 SELECT * FROM Customer;
108 SELECT * FROM Vehicle;
109 SELECT * FROM Service;
110 ----- UPDATE (modify existing record) -----
111 UPDATE Service
112 SET status = 'Completed'
113 WHERE service_id = 305;
114 ----- DELETE (remove a record) -----
115 DELETE FROM Customer
116 WHERE customer_id = 106;
117

```

The Query result table shows the following data:

	CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai

Screenshot 7 – Aggregate Functions



The screenshot shows the Oracle FreeSQL web interface. The SQL editor contains the following query:

```

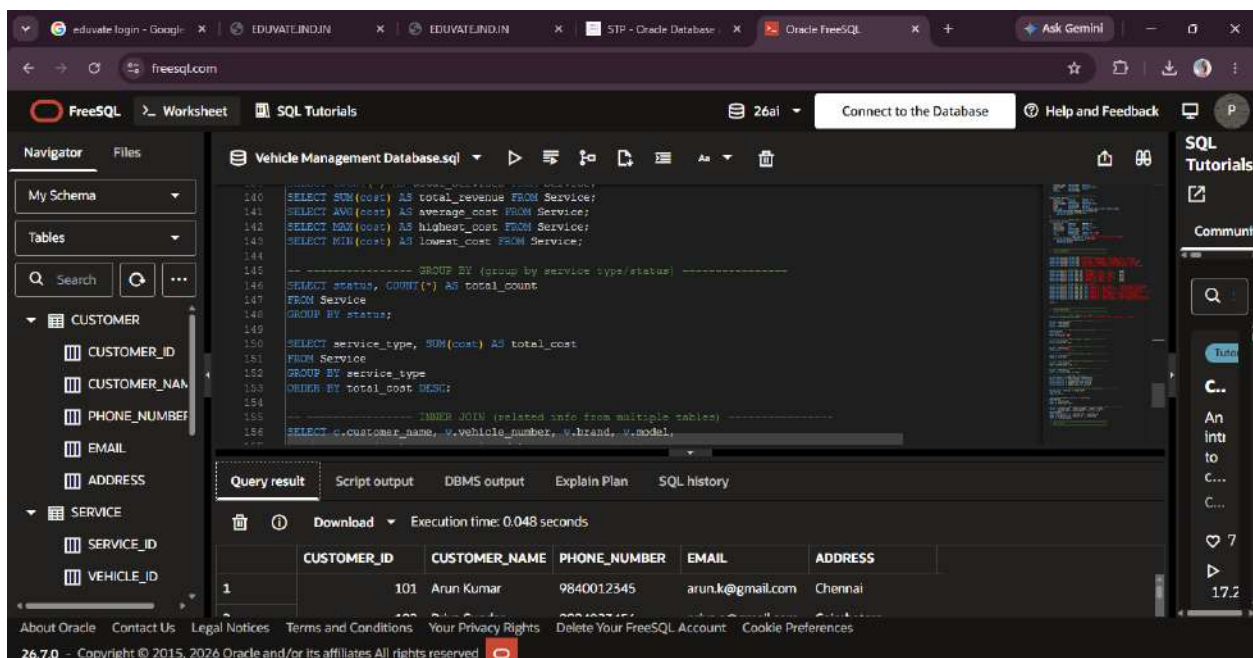
134 -- BETWEEN (range query)
135 SELECT * FROM Service
136 WHERE cost BETWEEN 1000 AND 2500;
137
138 ----- Aggregate Functions -----
139 SELECT COUNT(*) AS total_services FROM Service;
140 SELECT SUM(cost) AS total_revenue FROM Service;
141 SELECT AVG(cost) AS average_cost FROM Service;
142 SELECT MAX(cost) AS highest_cost FROM Service;
143 SELECT MIN(cost) AS lowest_cost FROM Service;
144
145 ----- GROUP BY (group by service type/status) -----
146 SELECT status, COUNT(*) AS total_count
147 FROM Service
148 GROUP BY status;
149
150 SELECT service_type, SUM(cost) AS total_cost
151 FROM Service

```

The query results are displayed in a table with the following data:

	CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai

Screenshot 8 – GROUP BY Operation



The screenshot shows the Oracle FreeSQL web interface. The SQL editor contains the following query:

```

140 SELECT SUM(cost) AS total_revenue FROM Service;
141 SELECT AVG(cost) AS average_cost FROM Service;
142 SELECT MAX(cost) AS highest_cost FROM Service;
143 SELECT MIN(cost) AS lowest_cost FROM Service;
144
145 ----- GROUP BY (group by service type/status) -----
146 SELECT status, COUNT(*) AS total_count
147 FROM Service
148 GROUP BY status;
149
150 SELECT service_type, SUM(cost) AS total_cost
151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;
154
155 ----- INNER JOIN (related info from multiple tables) -----
156 SELECT c.customer_name, v.vehicle_number, v.brand, v.model,

```

The query results are displayed in a table with the following data:

	CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai

Screenshot 9 – INNER JOIN Operation

The screenshot shows the Oracle FreeSQL web interface. The SQL editor contains the following query:

```

151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;
154
155 ----- INNER JOIN (related info from multiple tables) -----
156 SELECT s.customer_name, v.vehicle_number, v.brand, v.model,
157        s.service_type, s.service_date, s.cost, s.status
158 FROM Customer c
159 INNER JOIN Vehicle v ON c.customer_id = v.customer_id
160 INNER JOIN Service s ON v.vehicle_id = s.vehicle_id
161 ORDER BY s.service_date;
162
163 ----- END OF SCRIPT -----
164
165

```

The query results are displayed in a table with the following data:

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS	
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai

Execution time: 0.048 seconds

Screenshot 10 – Final Database / Query Results

The screenshot shows the Oracle FreeSQL web interface with a different query:

```

146 SELECT status, COUNT(*) AS total_count
147 FROM Service
148 GROUP BY status;
149
150 SELECT service_type, SUM(cost) AS total_cost
151 FROM Service
152 GROUP BY service_type
153 ORDER BY total_cost DESC;

```

The query results are displayed in a table with the following data:

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS	
1	101	Arun Kumar	9840012345	arun.k@gmail.com	Chennai
2	102	Priya Sundar	9884023456	priya.s@gmail.com	Coimbatore
3	103	Vignesh Raja	9994034567	vignesh.r@gmail.com	Madurai
4	104	Deepa Lakshmi	9791045678	deepa.l@gmail.com	Trichy
5	105	Karthik Subramani	8608056789	karthik.s@gmail.com	Salem
6	106	Meena Rani	9500067890	meena.r@gmail.com	Erode

Execution time: 0.048 seconds

5. Technical Explanation – 5 Marks

Database Design

Three separate tables were created — Customer, Vehicle, and Service — because each represents a distinct real-world entity with its own attributes. Separating them avoids repeating customer details for every vehicle and avoids repeating vehicle details for every service visit. This is normalization: each fact is stored once, and related tables are linked through keys instead of duplicating data.

Keys and Constraints

Each table has a single-column primary key (`customer_id`, `vehicle_id`, `service_id`) that uniquely identifies every row. Vehicle references Customer through the `customer_id` foreign key, and Service references Vehicle through the `vehicle_id` foreign key, enforcing that a service record cannot exist for a vehicle that isn't registered, and a vehicle cannot exist for a customer that isn't registered. NOT NULL constraints ensure essential fields are always provided, UNIQUE constraints prevent duplicate phone numbers or vehicle numbers, and a DEFAULT value keeps every new service marked 'Pending' until updated.

SQL Operations

INSERT was used to add new customers, vehicles, and service records. SELECT was used to retrieve and verify stored data. UPDATE was used to change the service's status once completed. DELETE was used to remove records that are no longer needed, with ON DELETE CASCADE ensuring child related records are cleaned up automatically.

Data Retrieval

WHERE was used to filter rows by a condition (e.g., pending services). ORDER BY sorted results by cost. LIKE searched for customers whose names matched a pattern. BETWEEN the services retrieved within a date range. GROUP BY combined with COUNT summarized how many services fall under each status.

Aggregate Functions and Joins

Aggregate functions (COUNT, SUM, AVG, MAX, MIN) were used to summarize the Service table — for example, calculating total revenue and the most/least expensive service. INNER JOIN was used to combine Customer, Vehicle, and Service into a single result set, so that a customer's name, their vehicle number, and the services performed on that vehicle could all be viewed together in one query.

6. Conclusion – 5 Marks

Conclusion

This project developed a Vehicle Service Management Database using Oracle to organize customer, vehicle, and service information that would otherwise be scattered and hard to track. Three related tables were designed with appropriate primary and foreign keys, sample data was inserted, and all core SQL operations were implemented and tested — including INSERT, SELECT, UPDATE, DELETE, filtering with WHERE/LIKE/BETWEEN, sorting with ORDER BY, summarizing with aggregate functions and GROUP BY, and combining tables with INNER JOIN. Through this project, key relational database concepts such as normalization, referential integrity, and multi-table querying were learned and applied practically.

Future Enhancements

- Customer login and authentication.
- Online service appointment booking.
- Automated service reminders.
- Service history reports.
- Invoice generation.
- Mechanic assignment and management.
- Integration with a web or mobile application.
- Automated database backup and recover.